

ADD-SLAM: Adaptive Dynamic Dense SLAM with Gaussian Splatting

Wenhua Wu Chenpeng Su Siting Zhu Tianchen Deng Zhe Liu Hesheng Wang *
Shanghai Jiao Tong University

Abstract

Recent advancements in Neural Radiance Fields (NeRF) and 3D Gaussian-based Simultaneous Localization and Mapping (SLAM) methods have demonstrated exceptional localization precision and remarkable dense mapping performance. However, dynamic objects introduce critical challenges by disrupting scene consistency, leading to tracking drift and mapping artifacts. Existing methods that employ semantic segmentation or object detection for dynamic identification and filtering typically rely on predefined categorical priors, while discarding dynamic scene information crucial for robotic applications such as dynamic obstacle avoidance and environmental interaction. To overcome these challenges, we propose ADD-SLAM: an Adaptive Dynamic Dense SLAM framework based on Gaussian splitting. We design an adaptive dynamic identification mechanism grounded in scene consistency analysis, comparing geometric and textural discrepancies between real-time observations and historical maps. Ours requires no predefined semantic category priors and adaptively discovers scene dynamics. Precise dynamic object recognition effectively mitigates interference from moving targets during localization. Furthermore, we propose a dynamic-static separation mapping strategy that constructs a temporal Gaussian model to achieve online incremental dynamic modeling. Experiments conducted on multiple dynamic datasets demonstrate our method’s flexible and accurate dynamic segmentation capabilities, along with state-of-the-art performance in both localization and mapping.

1 Introduction

Dense visual Simultaneous Localization and Mapping (SLAM) is the foundation of perception, navigation, and planning that finds wide applications in areas such as autonomous driving, mobile robotics, and virtual reality [1, 2].

Although existing dense visual SLAM methods based on Neural Radiance Field (NeRF) and 3D Gaussians have shown promising results in static scenes, their performance often deteriorates in complex dynamic environments due to the disruption caused by dynamic objects to the scene’s consistency. Dynamic SLAM faces two key challenges. The first is how to identify dynamic objects. Some methods predominantly rely on pre-trained detection, semantic segmentation, or optical flow networks for dynamic object identification [3, 4, 5, 6]. They suffer from false detections, generalization issues, and the need for predefined dynamic classes. Even if the objects are static, they will be separated mistakenly due to their class. DG-slam [7] designs a multi-view depth warp mask to compensate for missing objects. However, the occlusion caused by the view change is also contained in the mask and cannot be distinguished. Gassidy [8] performs instance segmentation of the scene and relies on object-by-object iterative analysis to distinguish dynamics. The latest WildGS-SLAM [9] introduces an uncertainty-aware approach that eliminates dependency on prior.

*Corresponding authors.

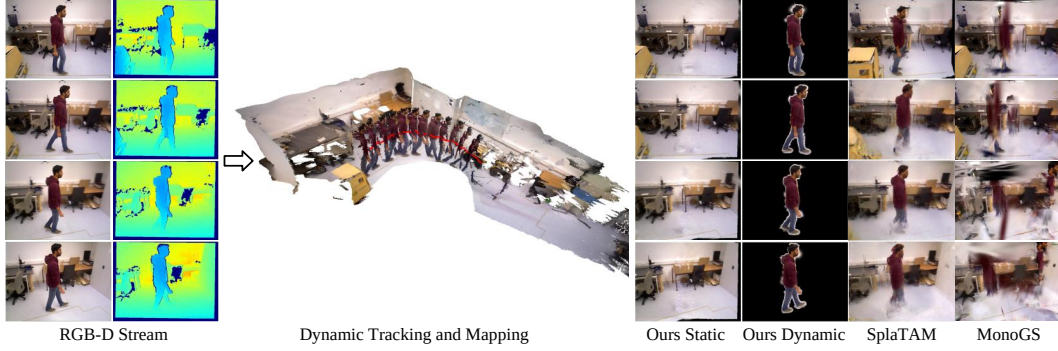


Figure 1: ADD-SLAM. Given RGB-D stream, our method achieves precise camera pose tracking while constructing dynamic-static composition maps. Our method can adaptively segment dynamic objects of any category without any semantic priors. The illustration presents effective dynamic tracking and mapping results, and high-quality rendering results of the dynamic-static separation.

However, **uncertainties in dynamic object boundaries remain prone to ambiguity**. Additionally, when the frame sequence is short, the uncertainty network cannot be trained sufficiently, resulting in significantly weaker capabilities to handle dynamic objects.

The second challenge is **how to track and model dynamic objects**. Dynamic information is crucial for the perception and interaction of intelligent robots. However, existing dynamic SLAM methods typically filter out dynamic objects, focusing solely on constructing static maps. Although some offline dynamic modeling methods [10, 11, 12] exist, they are time-consuming and require camera poses input, making them unsuitable for SLAM systems.

To address the aforementioned challenges, we propose ADD-SLAM, an adaptive dynamic dense visual SLAM with Gaussian splatting. Unlike existing approaches that rely on priors or uncertainty perception, we design an adaptive dynamic identification mechanism grounded in scene consistency analysis, comparing geometric and textural discrepancies between real-time observations and historical maps. In static scenes, observations from different viewpoints exhibit consistency. When objects move, this consistency is disrupted. We identify dynamic regions by detecting inconsistencies between the rendered images and actual observations and then obtain fine-grained dynamic segmentation using MobileSAM [13]. Our method does not require predefined dynamic object categories and can adaptively detect dynamics directly from the scene, offering greater flexibility.

Building on accurate dynamic object identification, we continuously perform 2D tracking of dynamic objects. During camera tracking, dynamic objects are excluded to prevent interference, resulting in more accurate localization. In the mapping process, we design a dynamic-static composition mapping method. When an object is first identified as dynamic, it is removed from the static map. The occluded background is gradually filled during the object’s motion. For dynamic objects, we construct a temporal Gaussian model to achieve online incremental dynamic modeling.

Overall, the main contributions of this paper are as follows:

- We propose a novel adaptive dynamic dense visual SLAM, ADD-SLAM, which accurately performs tracking and mapping in complex dynamic environments while modeling dynamic objects. ADD-SLAM achieves promising performance across multiple dynamic datasets.
- We propose a scene consistency analysis-based dynamic identification method that enables **category-agnostic adaptive recognition of dynamic objects without requiring semantic priors**, which is more accurate and flexible.
- We design a hybrid mapping approach that combines static and dynamic components, utilizing a temporal Gaussian model to achieve online incremental dynamic modeling.

2 Related Work

Traditional Visual SLAM. The foundation of dense visual SLAM lies in DTAM [14], which pioneers real-time tracking and mapping via dense scene representation. In the same period, the KinectFusion method [15] makes notable strides by using ICP algorithms [16] and volumetric TSDF to achieve

accurate and real-time reconstruction of dense surfaces for indoor scenes. Lots of innovative data structures including Surfels [17, 18] and Octrees [19, 20], are proposed to improve scalability and reduce memory. In contrast to these methods which rely on per-frame pose optimization, BAD-SLAM [21] is the first to propose a full Bundle Adjustment (BA) to jointly optimize the keyframes. Recently, numerous deep learning-based SLAM methods [22, 23, 24, 25, 26] are introduced to improve the precision and robustness of traditional SLAM methods with various learnable parameters.

NeRF and 3DGS based SLAM. Recent advances in Neural Radiance Fields (NeRF) [27] and 3D Gaussian splatting (3DGS) [28] have demonstrated remarkable success in SLAM, particularly due to their flexible scene parameterization and photorealistic rendering capabilities. iMAP [29] pioneered the integration of NeRF into SLAM, achieving real-time joint optimization, though a single MLP setup often limits reconstruction detail and leads to catastrophic forgetting. This challenge inspired NICE-SLAM [30] to incorporate a hierarchical scene representation, improving scalability and efficiency. Subsequent studies [31, 32, 33, 34, 35, 36, 37, 38, 39, 40] have made a series of improvements in scene representation and camera tracking. Considering the higher rendering quality and faster rendering speed of 3DGS, recent research has turned to using 3D Gaussians as map representations [41, 42, 43, 44, 45]. However, they assume that the scene is static. The disruption of scene consistency by dynamic objects can lead to significant performance degradation or even complete failure.

SLAM in Dynamic Environments. To handle dynamics, several methods [5, 46, 4] employ semantic segmentation or object detection. However, they rely on predefined dynamic category priors, leading to two inherent limitations: (1) inability to process unknown object categories, and (2) misclassification of stationary objects from predefined dynamic categories as moving objects. DynaMoN [3] and RoDyn-SLAM [47] incorporate optical flow estimation, but the inherent ambiguity between object motion and camera motion measurements remains challenging to disambiguate. DG-SLAM [7] designs a multi-view depth warp mask to compensate for missing objects. However, the occlusion caused by the view change is contained in the mask. Gassidy [8] performs instance segmentation of the scene and relies on object-by-object iterative analysis to distinguish dynamics. The latest WildGS-SLAM [9] introduces an uncertainty-aware approach that eliminates dependency on prior. However, uncertainties in dynamic object boundaries remain prone to ambiguity, and the incrementally trained MLP performs poorly at the beginning stage and short sequence case. In contrast, our method adaptively segments arbitrary dynamic objects through scene consistency analysis, requiring no prior knowledge while achieving precise boundary delineation. Furthermore, whereas existing methods simply filter out dynamic elements to construct static maps only, ours simultaneously builds both dynamic and static maps.

3 Method

Our proposed adaptive dynamic dense SLAM framework is illustrated in Fig. 2. The input consists of an RGB-D image sequence $\{(I_t, D_t) | t = 0, 1, \dots, n\}$ and camera intrinsic parameter K , while the output of ADD-SLAM includes the camera poses and the dynamic-static composition map. The process begins with **static map initialization using the first frame** (3.1). Next, dynamic objects in the environment are adaptively segmented based on dynamic consistency analysis, and a dynamic tracking sequence is constructed (3.2). Building upon this, dynamic-static separation is performed on the original static map. Camera pose optimization is then conducted using tracking loss (3.3), followed by dynamic-static composition mapping (3.4).

3.1 3D Gaussian Splatting

Our dynamic SLAM employs 3D Gaussian as the static map representation. Following previous 3D Gaussian-based SLAM methods, we initialize the static map using the first frame. The first RGB-D images are reconstructed into a point cloud based on the intrinsic parameters and the initial pose. The point cloud is then initialized as 3D Gaussian ellipsoid $G_s = \{(\mu_i, \Sigma_i, o_i, h_i)\}$. Each Gaussian ellipsoid contains the center position $\mu_i \in \mathbb{R}^3$, covariance matrix $\Sigma_i \in \mathbb{R}^{3 \times 3}$, opacity $o \in \mathbb{R}$, and spherical harmonic coefficients $h \in \mathbb{R}^{3(L+1)^2}$, where L is the order of the spherical harmonics. The Gaussian function $g_i(x)$ is:

$$g_i(x) = e^{-\frac{1}{2}(x-\mu_i)^T \Sigma_i^{-1}(x-\mu_i)}. \quad (1)$$

occluded background has been exposed. The new occlusion caused by the dynamic object in the background will make the observed depth smaller than the rendered depth. In contrast, the newly exposed background region will have an observed depth larger than the rendered depth. This allows for the distinction between dynamic objects and the background.

$$M_{ic}^d = M_{ic} \cap ((D - \hat{D}) < 0). \quad (5)$$

Dynamic Fine Segmentation. We utilize MobileSAM [13] to perform detailed segmentation on dynamic objects, which is a vision foundation model for segmentation guided by prompts. Due to the minimal object movement between consecutive frames (with a time interval of 1/30s), the resulting dynamic inconsistency region represents only a small portion of the object. Therefore, we use the center of the inconsistent region as a prompt input for MobileSAM f_θ to obtain the complete dynamic object segmentation.

$$M_d = f_\theta(I, o(M_{ic}^d)), \quad (6)$$

where $o(\cdot)$ represents the center of the inscribed circle.

Dynamic Object Tracking. Building upon the identification of dynamic objects in the scene, we construct a 2D dynamic tracking sequence. Each dynamic object is assigned a unique id along with its mask in each frame and the object center, defined as the center of the inscribed circle within the mask. When the next frame is processed, previously recognized objects do not require inconsistency re-detection. Instead, the object center serves as a prompt for MobileSAM to generate an updated mask and refine the object center, enabling continuous object tracking.

$$M_d^{t+1} = f_\theta(I_{t+1}, o(M_d^t)). \quad (7)$$

For dynamic objects initially present within the field of view rather than those entering from outside, We backtrack the frames before the detection of the dynamics, ensuring temporal-coherent segmentation across all frames.

Unlike traditional object detection methods that operate on single frames and ignore temporal consistency, our approach incorporates dynamic tracking to naturally associate objects across frames, effectively capturing their motion over time. In contrast to existing dynamic segmentation methods, which often depend on category-specific pre-trained detection networks, our method identifies dynamic objects by detecting scene inconsistencies caused by motion. It is entirely category-agnostic and does not rely on any pre-trained detectors. By leveraging the MobileSAM visual foundation model, we achieve fine-grained segmentation of dynamic objects, regardless of their category.

3.3 Camera Tracking

For each incoming frame, we employ frame-to-model tracking. The camera pose is initialized based on the constant velocity assumption. Then, it is refined using rendering loss with well-reconstructed static regions.

$$M_{track} = (\neg M_d) \cap (\hat{O} > \tau_{track}), \quad (8)$$

$$T_t = \arg \min \lambda_{track} \sum M_{track} \cdot \|\hat{I} - I\|_1 + (1 - \lambda_{track}) \sum (M_{track} \cap M_v) \cdot \|\hat{D} - D\|_1, \quad (9)$$

where τ_{track} represents the opacity threshold for the tracking region. M_v is the valid region of the ground truth depth, taking into account the presence of holes in the depth ground truth.

To mitigate pose drift in extended sequences, we incorporate loop detection and global bundle adjustment (BA). Following Droid-SLAM [26], we integrate a pre-trained optical flow model with Dense Bundle Adjustment Layer (DBA) to optimize keyframe camera poses and depth. In contrast to WildGS-SLAM [9] that introduces uncertainty maps during BA optimization, we leverage acquired dynamic masks to eliminate interference from moving entities while preserving static scene integrity. Following DG-SLAM [7], the cost function over the keyframe graph is defined as:

$$\mathbf{E}(\mathbf{T}, \mathbf{d}) = \sum_{(i,j) \in \mathcal{E}} \|\mathbf{p}_{ij}^* - \Pi_c(\mathbf{T}_{ij} \circ \Pi_c^{-1}(\mathbf{p}_i, \mathbf{d}_i))\|_{\Sigma_{ij}, \neg M_d}^2, \quad \Sigma_{ij} = \text{diag } \omega_{ij}, \quad (10)$$

where $\mathbf{p}_{ij}^*$ where Π_c denotes the projection transformation from 3D coordinates to the image plane. $\mathbf{p}_i$ represents pixel coordinates, $\mathbf{d}_i$ indicates inverse depth values. $\mathbf{T}_{ij}$ corresponds to the relative camera pose between frames i and j . $\mathbf{p}_{ij}^*$ represents the propagated coordinates of pixel $\mathbf{p}_i$ in frame j through optical flow estimation. $\|\cdot\|_{\Sigma_{ij}, \neg M_d}^2$ denotes the Mahalanobis distance weighted by confidence metric Σ_{ij} , while filtering the dynamic.

3.4 Dynamic-static Composite Mapping

Unlike existing dynamic SLAM [7, 9, 8] that perform elimination of dynamic objects, we propose a dynamic-static composite mapping strategy that leverages temporal Gaussian models to achieve online incremental dynamic modeling.

Dynamic-Static Separation. At the start of ADD-SLAM, we initialize a completely static map. Once the detection module identifies dynamic objects, we immediately separate them from the static map. Specifically, we use **MobileSAM to segment dynamic objects from the rendered images**, thereby determining their 2D positions within the map. Then, using the depth map and camera intrinsic and extrinsic parameters, we obtain the point cloud of the dynamic objects. In the original static Gaussian map, we filter out the Gaussian spheres representing the dynamic object regions, thus achieving adaptive dynamic-static separation.

Static Mapping. For each newly added keyframe, we insert Gaussian ellipsoids into the static Gaussian map to fill regions with holes or poor quality in the rendered output.

$$M_{inert} = (\neg M_d) \cap M_v \cap (\hat{O} < \tau_{map}), \quad (11)$$

where τ_{map} represents the opacity threshold for the static mapping region. The RGB-D pixels in M_{in} will be initialized as Gaussian ellipsoids and incorporated into the static Gaussian map. Subsequently, keyframes are selected from the keyframe set to optimize the static map. We compute the rendering loss for the valid static regions:

$$L_I = (1 - \lambda_{ssim}) \frac{1}{M_{map}} \sum M_{map} \cdot \|\hat{I} - I\|_1 + \lambda_{ssim} SSIM(\hat{I}, I, M_{map}), \quad (12)$$

$$L_D = \frac{1}{M_{map}} \sum M_{map} \cdot \|\hat{D} - D\|_1, \quad M_{map} = (\neg M_d) \cap M_v, \quad (13)$$

where λ_{ssim} is the weight of ssim loss. The final static mapping loss is:

$$L_{map}^{static} = \lambda_{color} L_I + \lambda_{depth} L_D + \lambda_{reg} L_{reg}, \quad (14)$$

where L_{reg} is the Gaussian ellipsoid scale regularization loss from [48].

Dynamic Mapping. For each tracked dynamic object id , we construct a temporal Gaussian model:

$$G_d^{id}(t) = \{(\mu_i^t, \Sigma_i^t, o_i^t, h_i^t)\}. \quad (15)$$

Each Gaussian ellipsoid at a given time is initialized using the corresponding 2D tracking results. At time t , the corresponding dynamic Gaussians $\{G_d^{id}(t), id \in ID\}$ are used to render the dynamic image:

$$\hat{I}_d = \sum_{i \in N_d} c_i^t \alpha_i^t \prod_{j=1}^{i-1} (1 - \alpha_j^t), \quad \hat{D}_d = \sum_{i \in N_d} d_i^t \alpha_i^t \prod_{j=1}^{i-1} (1 - \alpha_j^t). \quad (16)$$

Then, the dynamic Gaussians are optimized using the rendering loss:

$$L_I^d = (1 - \lambda_{ssim}) \frac{1}{M_{map}^d} \sum M_{map}^d \cdot \|\hat{I}_d - I\|_1 + \lambda_{ssim} SSIM(\hat{I}_d, I, M_{map}^d), \quad (17)$$

$$L_D^d = \frac{1}{M_{map}^d} \sum M_{map}^d \cdot \|\hat{D}_d - D\|_1, \quad M_{map}^d = M_d \cap M_v. \quad (18)$$

The final dynamic mapping loss is:

$$L_{map}^d = \lambda_{color} L_I^d + \lambda_{depth} L_D^d + \lambda_{reg} L_{reg}. \quad (19)$$

4 Experiment

4.1 Experimental Setting

Dataset. We evaluate ADD-SLAM on three real-world dynamic datasets: the TUM RGB-D [54] dataset, the Bonn dataset [55], and the DAVIS dataset [56].

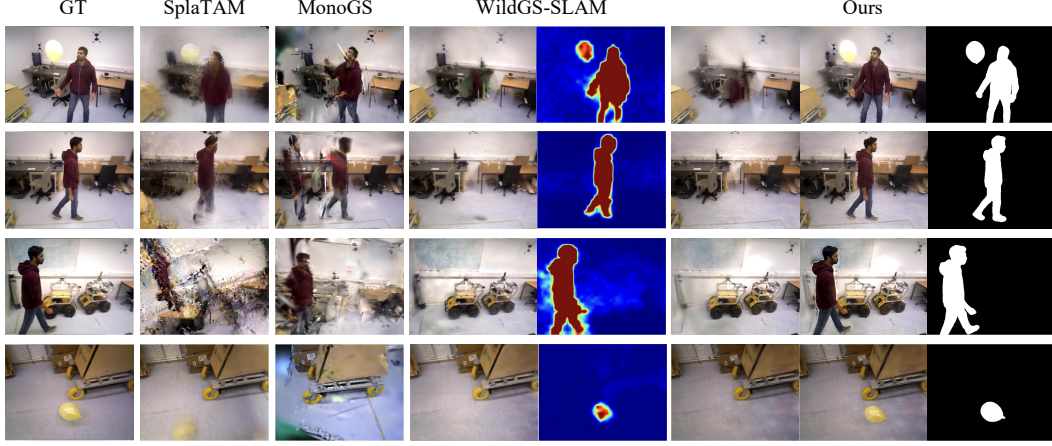


Figure 3: Rendering Visualization. Compared to other methods, ADD-SLAM not only accurately reconstructs the static background but also captures fine details of the dynamic foreground. Our dynamic mask is more complete and precise than the uncertainty of WildGS-SLAM [9].

Table 1: Camera tracking results on Bonn dataset. "*" denotes the version reproduced by NICE-SLAM. "-" denotes the absence of mention. "X" denotes a failure in execution, with no valid result. "†" indicates the replacement of depth estimation with ground-truth depth. The metric unit is [cm]. Best results are highlighted as **first**, **second**, and **third**.

Methods	Input	balloon		balloon2		ps_track		ps_track2		ball_track		mv_box2		Avg.	
<i>Traditional</i>		<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>
ORB-SLAM3[49]	RGBD	5.8	2.8	17.7	8.6	70.7	32.6	77.9	43.8	3.1	1.6	3.5	1.5	29.8	15.2
Droid-VO[26]	RGBD	5.4	-	4.6	-	21.34	-	46.0	-	8.9	-	5.9	-	15.4	-
DynaSLAM[50]	RGBD	3.0	-	2.9	-	6.1	-	7.8	-	4.9	-	3.9	-	4.77	-
ReFusion[51]	RGBD	17.5	-	25.4	-	28.9	-	46.3	-	30.2	-	17.9	-	27.7	-
<i>NeRF based</i>		<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>
iMAP*[29]	RGBD	14.9	5.4	67.0	19.2	28.3	12.9	52.8	20.9	24.8	11.2	28.3	35.3	36.1	17.5
NICE-SLAM[30]	RGBD	X	X	66.8	20.0	54.9	27.5	45.3	17.5	21.2	13.1	31.9	13.6	-	-
Vox-Fusion[33]	RGBD	65.7	30.9	82.1	52.0	128.6	52.5	162.2	46.2	43.9	16.5	47.5	19.5	88.4	36.3
Co-SLAM[31]	RGBD	28.8	9.6	20.6	8.1	61.0	22.2	59.1	24.0	38.3	17.4	70.0	25.5	46.3	17.8
ESLAM[32]	RGBD	22.6	12.2	36.2	19.9	48.0	18.7	51.4	23.2	12.4	6.6	17.7	7.5	31.4	14.7
RoDyn-SLAM[47]	RGBD	7.9	2.7	11.5	6.1	14.5	4.6	13.8	3.5	13.3	4.7	12.6	4.7	12.3	4.38
DynaMon(MS&SS)[3]	RGB	2.8	-	2.7	-	14.8	-	2.2	-	3.4	-	2.7	-	4.77	-
<i>3DGS based</i>		<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>
SplaTAM[44]	RGBD	40.0	14.6	39.5	15.8	217.9	81.2	131.0	33.1	20.2	16.3	17.1	9.3	77.6	28.4
MonoGS[45]	RGBD	31.2	15.3	26.7	13.5	43.8	16.8	48.4	16.6	4.7	2.4	7.1	3.5	27.0	11.4
GS-ICP SLAM[52]	RGBD	42.2	14.4	57.5	22.4	87.8	40.6	49.8	21.2	32.1	11.7	26.0	12.4	49.2	20.5
DG-SLAM[7]	RGBD	3.7	-	4.1	-	4.5	-	6.9	-	10.0	-	3.5	-	5.45	-
WildGS-SLAM[9]	RGB	2.9	1.2	2.5	1.2	3.6	1.9	3.1	1.4	3.1	1.6	2.4	1.3	2.93	1.43
WildGS-SLAM†[9]	RGBD	2.4	1.0	2.4	1.2	3.4	1.9	3.1	1.4	3.4	1.9	2.6	1.3	2.88	1.45
Ours	RGBD	2.7	1.1	2.3	0.8	2.4	1.1	3.7	1.4	3.4	1.2	2.1	0.8	2.77	1.05

Evaluation Metrics. For camera tracking performance, we use the Root Mean Square Error (RMSE) and Standard Deviation (S.D.) of the Absolute Trajectory Error (ATE). For the quality of Gaussian-based mapping, we employ image rendering quality metrics, including Peak Signal-to-Noise Ratio (PSNR), Structure Similarity Index Measure (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS). Since the original images contain both static backgrounds and dynamic foregrounds, the rendering metrics can effectively reflect the performance of dynamic-static composition mapping.

Baselines. We conduct extensive and comprehensive comparisons between ADD-SLAM and traditional SLAM methods, NeRF-based SLAM methods, and 3D Gaussian-based SLAM methods to highlight the superiority of our method.

Implementation Details. The order of spherical harmonics L is 0. The thresholds for color and geometric inconsistencies $\tau_I = 20 \cdot \text{median}(I_{err})$ and $\tau_D = 20 \cdot \text{median}(D_{err})$. The opacity thresholds for tracking τ_{track} and mapping τ_{map} are set to 0.7 and 0.8, respectively. The weights of loss $\lambda_{track} = 0.6$, $\lambda_{ssim} = 0.2$, $\lambda_{color} = 1.0$, $\lambda_{depth} = 1.0$, $\lambda_{reg} = 1.0$. The iterative optimization steps for tracking and mapping are set to 100. Adaptive dynamic detection is performed every 5 frames for Bonn and DAVIS dataset, and every 10 frames for TUM RGB-D dataset. All experiments are conducted on a server equipped with a Intel Platinum 8362 CPU and an NVIDIA A100 GPU.

Table 2: Camera tracking results on TUM RGB-D dataset. "*" denotes the version reproduced by NICE-SLAM. "-" denotes the absence of mention. "X" denotes a failure in execution, with no valid result. The metric unit is [cm]. Best results are highlighted as **first**, **second**, and **third**.

Methods	Input	Dynamic								Static				Avg.	
		fr3/wk_xyz		fr3/wk_hf		fr3/wk_st		fr3/st_hf		fr1/xyz		fr1/rpy			
<i>Traditional</i>		<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>
ORB-SLAM3 [49]	RGBD	28.1	12.2	30.5	9.0	2.0	1.1	2.6	1.6	1.1	0.6	2.2	1.3	11.1	4.3
DVO-SLAM[53]	RGBD	59.7	-	52.9	-	21.2	-	6.2	-	1.1	-	2.0	-	22.9	-
DynaSLAM[50]	RGBD	1.7	-	2.6	-	0.7	-	2.8	-	-	-	-	-	-	-
ReFusion[51]	RGBD	9.9	-	10.4	-	1.7	-	11.0	-	-	-	-	-	-	-
<i>NeRF based</i>		<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>
iMAP*[29]	RGBD	111.5	43.9	X	X	137.3	21.7	93.0	35.3	7.9	7.3	16.0	13.8	-	-
NICE-SLAM[30]	RGBD	113.8	42.9	X	X	88.2	27.8	45.0	14.4	4.6	3.8	3.4	2.5	-	-
Vox-Fusion[33]	RGBD	146.6	32.1	X	X	109.9	25.5	89.1	28.5	1.8	0.9	4.3	3.0	-	-
Co-SLAM[31]	RGBD	51.8	25.3	105.1	42.0	49.5	10.8	4.7	2.2	2.3	1.2	3.9	2.8	36.3	14.1
ESLAM[32]	RGBD	45.7	28.5	60.8	27.9	93.6	20.7	3.6	1.6	1.1	0.6	2.2	1.2	34.5	13.5
RoDyn-SLAM[47]	RGBD	8.3	5.5	5.6	2.8	1.7	0.9	4.4	2.2	1.5	0.8	2.8	1.5	4.05	2.28
DynaMoN(MS&SS)[3]	RGB	1.4	-	1.9	-	0.7	-	2.3	-	-	-	-	-	-	-
<i>3DGS based</i>		<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>	<i>RMSE</i>	<i>S.D.</i>
SplaTAM[44]	RGBD	160.5	42.4	X	X	42.6	13.0	14.5	6.2	1.1	0.6	3.2	1.5	44.4	12.8
MonoGS[45]	RGBD	28.4	12.3	47.8	16.5	15.3	8.4	13.9	3.2	1.0	0.4	2.6	1.3	18.2	7.0
GS-ICP[52]	RGBD	68.9	50.8	84.6	34.3	87.5	16.9	11.2	2.7	1.4	0.7	4.2	3.9	43.0	18.2
DG-SLAM[7]	RGBD	1.6	-	0.6	-	-	-	-	-	-	-	-	-	-	-
WildGS-SLAM[9]	RGB	1.2	0.6	1.5	0.8	0.5	0.2	1.8	1.1	0.9	0.5	2.3	1.3	1.37	0.75
WildGS-SLAM†[9]	RGBD	1.2	0.6	1.4	0.7	0.5	0.2	1.7	1.0	0.9	0.5	2.2	1.0	1.32	0.67
Ours	RGBD	1.4	0.9	1.6	0.8	0.5	0.2	1.3	0.6	1.0	0.5	1.7	0.9	1.25	0.65

4.2 Experimental Results

Evaluation of Camera Tracking Performance. The camera tracking results on the Bonn and TUM RGB-D datasets are shown in Tab. 1 and Tab. 2. Note that the original WildGS-SLAM [9] uses RGB input with depth estimation. For fair comparison, we replace the estimated depth with ground truth depth, denoted as "†". Compared with various traditional methods, NeRF-based and 3DGS-based methods, **our method achieves state-of-the-art performance in both static and dynamic scenarios.**

Table 3: Rendering performance on Bonn dataset.

Methods	balloon			balloon2			ps_track			ps_track2			ball_track			mv_box2			Avg		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
SplaTAM[44]	16.92	0.78	0.22	17.35	0.69	0.30	17.11	0.62	0.33	15.54	0.58	0.37	19.83	0.80	0.22	20.93	0.85	0.18	17.95	0.72	0.27
MonoGS[45]	20.25	0.77	0.34	18.54	0.72	0.38	19.64	0.76	0.37	18.47	0.71	0.40	23.40	0.81	0.33	23.51	0.84	0.26	20.64	0.77	0.35
Ours	22.91	0.91	0.22	19.10	0.80	0.31	23.53	0.91	0.25	22.95	0.91	0.24	23.23	0.88	0.28	22.74	0.89	0.27	22.41	0.89	0.26

Evaluation of Mapping Performance. Unlike existing approaches that either neglect dynamic elements or directly filter them out for static mapping, our method achieves dynamic-static decoupled mapping. To evaluate comprehensive mapping performance, we perform dynamic-static combined rendering to assess the complete scene reconstruction $G_{all}(t) = G_s \cup \{G_d^{id}(t), id \in ID\}$. Tab. 3 presents quantitative evaluations of our mapping results, demonstrating significant superiority over state-of-the-art dense mapping methods. Fig. 3 provides visual comparisons of rendering quality: SplaTAM [44] and MonoGS [45] fail to handle dynamic elements, producing extensive blur and artifacts. Although WildGS-SLAM [9] employs uncertainty-aware dynamic removal, its ambiguous boundary delineation of uncertain dynamic objects leads to residual interference, exemplified by the incomplete foot segmentation in row 2, where ground plane residuals persist. In contrast, our method achieves precise dynamic object segmentation with clean background reconstruction. Furthermore, ours uniquely enables dynamic object modeling, a capability absent in comparative methods.

Evaluation of dynamic mask. Considering that the scenes and dynamic categories in the Bonn and TUM-RGBD datasets are limited, we conducted experiments on the dedicated dynamic segmentation dataset DAVIS, which contains richer scenes and dynamic objects. Fig. 4 shows a comparison between our dynamic mask and the uncertainty of WildGS-SLAM [9]. Our dynamic mask is more complete and precise than WildGS-SLAM's uncertainty, and it is not disturbed by the background. The dynamic segmentation of various objects in multiple environments demonstrates the adaptability and robustness of our method.

Table 4: Comparison of different methods in terms of running time (ms).

Methods	Dynamic Seg.	Tracking	Mapping
Rodyn-SLAM[47]	278.66	875.70	1083.60
SplaTAM[44]	-	2630.36	548.06
WildGS-SLAM[9]	25.77	2467.28	2948.27
Ours	68.79	1025.39	1108.78

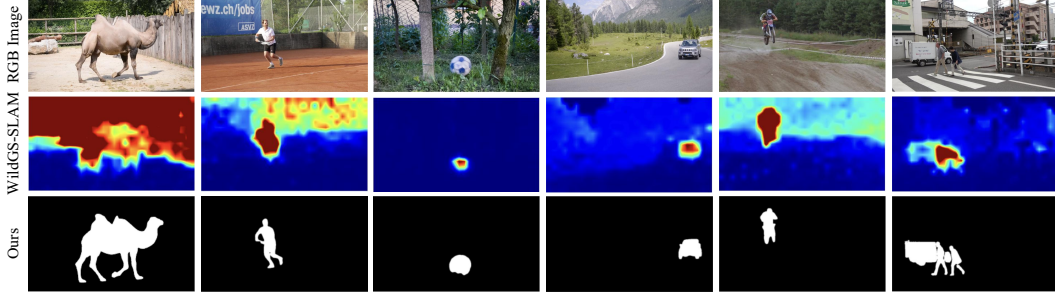


Figure 4: Comparison between our dynamic mask and the uncertainty of WildGS-SLAM [9] on the DAVIS dataset.

Runtime Analysis. Tab. 4 presents the runtime analysis, including the time for dynamic segmentation, camera tracking, and mapping. Our tracking and mapping times are comparable to other methods, while additionally performing dynamic mapping. For dynamic segmentation, our method is more efficient than Rodyn-SLAM[47], which relies on pre-trained semantic segmentation and optical flow networks.

Ablation Study We conduct an ablation study on our adaptive dynamic segmentation. Fig. 5 presents a comparison between our method’s dynamic segmentation results and those obtained by a semantic segmentation network. We utilize MaskDINO[57], with the predefined dynamic category set to “human”. Unlike prior-based semantic segmentation, our method can adaptively detect atypical moving objects, such as boxes and balloons, enabling more precise and flexible dynamic segmentation. Tab. 5 shows the camera tracking results under different settings: a. without dynamic segmentation, b. using the prior-based semantic segmentation, and d. employing our adaptive dynamic segmentation. With more accurate and flexible dynamic segmentation, camera tracking accuracy is significantly improved. Additionally, we validated the effect of keyframe DBA. As shown in row c. of Tab. 5, **keyframe DBA can significantly improve tracking accuracy.**

Table 5: Comparison of tracking metrics of different ways to manange dynamic segmentation.

Methods	balloon		ball_track		mv_box2	
	<i>RMSE</i>	<i>SD</i>	<i>RMSE</i>	<i>SD</i>	<i>RMSE</i>	<i>SD</i>
a. Ours w/o dynamic detection	52.5	21.6	15.2	5.2	18.3	5.4
b. Ours w/ MaskDINO[57]	5.5	2.1	11.4	4.6	12.5	3.9
c. Ours w/o keyframe DBA	3.3	1.0	6.9	3.9	7.4	3.0
d. ADD-SLAM (Ours)	2.7	1.0	3.4	1.2	2.1	0.8

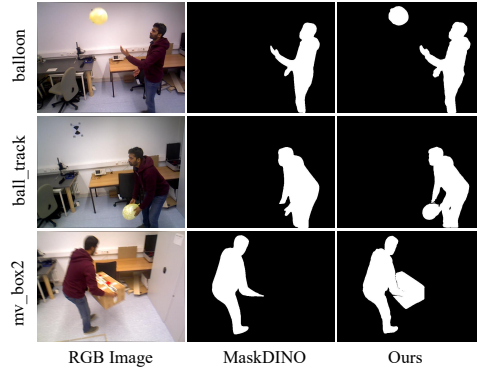


Figure 5: Comparison of the dynamic segmentation results of our method with those obtained using a semantic segmentation network.

5 Conclusion

We propose ADD-SLAM, a novel dynamic dense visual SLAM system. We introduce an adaptive dynamic object identification method based on scene consistency analysis, which eliminates the need for pre-trained detection or optical flow networks, offering greater accuracy and flexibility. Unlike existing methods that directly filter out dynamic objects, we design a hybrid mapping approach that models dynamic objects by constructing dynamic Gaussian sequences, preserving scene dynamics. Extensive experiments demonstrate the outstanding performance of our method, achieving not only accurate tracking but also realistic dynamic-static scene modeling. Our method provides a technical foundation for robotic localization and perception in complex dynamic environments. **Limitation:** Although our method has achieved remarkable performance, it **relies on mobileSAM to segment inconsistent regions**. Partial segmentation or over-segmentation in complex environments can negatively impact the performance.

References

- [1] Yanan Wang, Yaobin Tian, Jiawei Chen, Kun Xu, and Xilun Ding. A survey of visual slam in dynamic environment: the evolution from geometric to semantic approaches. *IEEE Transactions on Instrumentation and Measurement*, 2024.
- [2] Basheer Al-Tawil, Thorsten Hempel, Ahmed Abdelrahman, and Ayoub Al-Hamadi. A review of visual slam for robotics: evolution, properties, and future applications. *Frontiers in Robotics and AI*, 11:1347985, 2024.
- [3] Nicolas Schischka, Hannah Schieber, Mert Asim Karaoglu, Melih Gorgulu, Florian Grötzner, Alexander Ladikos, Nassir Navab, Daniel Roth, and Benjamin Busam. Dynamon: Motion-aware fast and robust camera localization for dynamic neural radiance fields. *IEEE Robotics and Automation Letters*, 2024.
- [4] Ziheng Xu, Jianwei Niu, Qingfeng Li, Tao Ren, and Chen Chen. Nid-slam: Neural implicit representation-based rgb-d slam in dynamic environments. *arXiv preprint arXiv:2401.01189*, 2024.
- [5] Chenyu Ruan, Qiuyu Zang, Kehua Zhang, and Kai Huang. Dn-slam: A visual slam with orb features and nerf mapping in dynamic environments. *IEEE Sensors Journal*, 2023.
- [6] Haoang Li, Xiangqi Meng, Xingxing Zuo, Zhe Liu, Hesheng Wang, and Daniel Cremers. Pg-slam: Photo-realistic and geometry-aware rgb-d slam in dynamic environments. *arXiv preprint arXiv:2411.15800*, 2024.
- [7] Yueming Xu, Haochen Jiang, Zhongyang Xiao, Jianfeng Feng, and Li Zhang. Dg-slam: Robust dynamic gaussian splatting slam with hybrid pose optimization. *arXiv preprint arXiv:2411.08373*, 2024.
- [8] Long Wen, Shixin Li, Yu Zhang, Yuhong Huang, Jianjie Lin, Fengjunjie Pan, Zhenshan Bing, and Alois Knoll. Gassidy: Gaussian splatting slam in dynamic environments. *arXiv preprint arXiv:2411.15476*, 2024.
- [9] Jianhao Zheng, Zihan Zhu, Valentin Bieri, Marc Pollefeys, Songyou Peng, and Iro Armeni. Wildgs-slam: Monocular gaussian splatting slam in dynamic environments. *arXiv preprint arXiv:2504.03886*, 2025.
- [10] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis. In *2024 International Conference on 3D Vision (3DV)*, pages 800–809. IEEE, 2024.
- [11] Guanjuan Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4d gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20310–20320, 2024.
- [12] Zeyu Yang, Hongye Yang, Zijie Pan, and Li Zhang. Real-time photorealistic dynamic scene representation and rendering with 4d gaussian splatting. In *The Twelfth International Conference on Learning Representations*.
- [13] Chaoning Zhang, Dongshen Han, Yu Qiao, Jung Uk Kim, Sung-Ho Bae, Seungkyu Lee, and Choong Seon Hong. Faster segment anything: Towards lightweight sam for mobile applications. *arXiv preprint arXiv:2306.14289*, 2023.
- [14] Richard A Newcombe, Steven J Lovegrove, and Andrew J Davison. Dtam: Dense tracking and mapping in real-time. In *2011 international conference on computer vision*, pages 2320–2327. IEEE, 2011.
- [15] Richard A Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. Kinectfusion: Real-time dense surface mapping and tracking. In *2011 10th IEEE international symposium on mixed and augmented reality*, pages 127–136. Ieee, 2011.

- [16] Paul J Besl and Neil D McKay. Method for registration of 3-d shapes. In *Sensor fusion IV: control paradigms and data structures*, volume 1611, pages 586–606. Spie, 1992.
- [17] Thomas Whelan, Stefan Leutenegger, Renato Salas-Moreno, Ben Glocker, and Andrew Davison. Elasticfusion: Dense slam without a pose graph. Robotics: Science and Systems, 2015.
- [18] Thomas Schöps, Torsten Sattler, and Marc Pollefeys. Surfelmeshing: Online surfel-based mesh reconstruction. *IEEE transactions on pattern analysis and machine intelligence*, 42(10):2494–2507, 2019.
- [19] Emanuele Vespa, Nikolay Nikolov, Marius Grimm, Luigi Nardi, Paul HJ Kelly, and Stefan Leutenegger. Efficient octree-based volumetric slam supporting signed-distance and occupancy mapping. *IEEE Robotics and Automation Letters*, 3(2):1144–1151, 2018.
- [20] Binbin Xu, Wenbin Li, Dimos Tzoumanikas, Michael Bloesch, Andrew Davison, and Stefan Leutenegger. Mid-fusion: Octree-based object-level multi-instance dynamic slam. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 5231–5237. IEEE, 2019.
- [21] Thomas Schops, Torsten Sattler, and Marc Pollefeys. Bad slam: Bundle adjusted direct rgb-d slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 134–144, 2019.
- [22] Michael Bloesch, Jan Czarnowski, Ronald Clark, Stefan Leutenegger, and Andrew J Davison. Codeslam—learning a compact, optimisable representation for dense visual slam. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2560–2568, 2018.
- [23] Ruihao Li, Sen Wang, and Dongbing Gu. Deepslam: A robust monocular slam system with unsupervised deep learning. *IEEE Transactions on Industrial Electronics*, 68(4):3577–3587, 2020.
- [24] Lukas Koestler, Nan Yang, Niclas Zeller, and Daniel Cremers. Tandem: Tracking and dense mapping in real-time using deep multi-view stereo. In *Conference on Robot Learning*, pages 34–45. PMLR, 2022.
- [25] Songyou Peng, Michael Niemeyer, Lars Mescheder, Marc Pollefeys, and Andreas Geiger. Convolutional occupancy networks. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part III 16*, pages 523–540. Springer, 2020.
- [26] Zachary Teed and Jia Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. *Advances in neural information processing systems*, 34:16558–16569, 2021.
- [27] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.
- [28] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.
- [29] Edgar Sucar, Shikun Liu, Joseph Ortiz, and Andrew J Davison. imap: Implicit mapping and positioning in real-time. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6229–6238, 2021.
- [30] Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R Oswald, and Marc Pollefeys. Nice-slam: Neural implicit scalable encoding for slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12786–12796, 2022.
- [31] Hengyi Wang, Jingwen Wang, and Lourdes Agapito. Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13293–13302, 2023.
- [32] Mohammad Mahdi Johari, Camilla Carta, and François Fleuret. Eslam: Efficient dense slam system based on hybrid representation of signed distance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17408–17419, 2023.

- [33] Xingrui Yang, Hai Li, Hongjia Zhai, Yuhang Ming, Yuqian Liu, and Guofeng Zhang. Vox-fusion: Dense tracking and mapping with voxel-based neural implicit representation. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 499–507. IEEE, 2022.
- [34] Youmin Zhang, Fabio Tosi, Stefano Mattoccia, and Matteo Poggi. Go-slam: Global optimization for consistent 3d instant reconstruction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3727–3737, October 2023.
- [35] Xinyang Liu, Yijin Li, Yanbin Teng, Hujun Bao, Guofeng Zhang, Yinda Zhang, and Zhaopeng Cui. Multi-modal neural radiance field for monocular dense slam with a light-weight tof sensor. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1–11, 2023.
- [36] Mingrui Li, Jiaming He, Yangyang Wang, and Hongyu Wang. End-to-end rgb-d slam with multi-mlps dense neural implicit representations. *IEEE Robotics and Automation Letters*, 2023.
- [37] Yuhang Ming, Weicai Ye, and Andrew Calway. idf-slam: End-to-end rgb-d slam with neural implicit mapping and deep feature tracking. *arXiv preprint arXiv:2209.07919*, 2022.
- [38] Haocheng Wang, Yanlong Cao, Xiaoyao Wei, Yejun Shou, Lingfeng Shen, Zhijie Xu, and Kai Ren. Structerf-slam: Neural implicit representation slam for structural environments. *Computers & Graphics*, 119:103893, 2024.
- [39] Wenhua Wu, Guangming Wang, Ting Deng, Sebastian Aegidius, Stuart Shanks, Valerio Modugno, Dimitrios Kanoulas, and Hesheng Wang. Dvn-slam: dynamic visual neural slam based on local-global encoding. *arXiv preprint arXiv:2403.11776*, 2024.
- [40] Tianchen Deng, Guole Shen, Tong Qin, Jianyu Wang, Wentao Zhao, Jingchuan Wang, Danwei Wang, and Weidong Chen. Plgslam: Progressive neural scene representation with local to global bundle adjustment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19657–19666, 2024.
- [41] Chi Yan, Delin Qu, Dan Xu, Bin Zhao, Zhigang Wang, Dong Wang, and Xuelong Li. Gs-slam: Dense visual slam with 3d gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19595–19604, 2024.
- [42] Huajian Huang, Longwei Li, Hui Cheng, and Sai-Kit Yeung. Photo-slam: Real-time simultaneous localization and photorealistic mapping for monocular stereo and rgb-d cameras. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21584–21593, 2024.
- [43] Vladimir Yugay, Yue Li, Theo Gevers, and Martin R Oswald. Gaussian-slam: Photo-realistic dense slam with gaussian splatting. *arXiv preprint arXiv:2312.10070*, 2023.
- [44] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. Splatam: Splat track & map 3d gaussians for dense rgb-d slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21357–21366, 2024.
- [45] Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian Splatting SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [46] Mingrui Li, Jiaming He, Guangan Jiang, and Hongyu Wang. Ddn-slam: Real-time dense dynamic neural implicit slam with joint semantic encoding. *arXiv preprint arXiv:2401.01545*, 2024.
- [47] Haochen Jiang, Yueming Xu, Kejie Li, Jianfeng Feng, and Li Zhang. Rodyn-slam: Robust dynamic dense rgb-d slam with neural radiance fields. *IEEE Robotics and Automation Letters*, 2024.
- [48] Liyuan Zhu, Yue Li, Erik Sandström, Shengyu Huang, Konrad Schindler, and Iro Armeni. Loopsplat: Loop closure by registering 3d gaussian splats. *arXiv preprint arXiv:2408.10154*, 2024.

- [49] Carlos Campos, Richard Elvira, Juan J Gómez Rodríguez, José MM Montiel, and Juan D Tardós. Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.
- [50] Berta Bescos, José M Fàcil, Javier Civera, and José Neira. Dynaslam: Tracking, mapping, and inpainting in dynamic scenes. *IEEE Robotics and Automation Letters*, 3(4):4076–4083, 2018.
- [51] Emanuele Palazzolo, Jens Behley, Philipp Lottes, Philippe Giguere, and Cyrill Stachniss. Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7855–7862. IEEE, 2019.
- [52] Seongbo Ha, Jiung Yeon, and Hyeonwoo Yu. Rgb-d gs-icp slam. In *European Conference on Computer Vision*, pages 180–197. Springer, 2025.
- [53] Christian Kerl, Jürgen Sturm, and Daniel Cremers. Dense visual slam for rgb-d cameras. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2100–2106. IEEE, 2013.
- [54] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of rgb-d slam systems. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 573–580. IEEE, 2012.
- [55] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss. ReFusion: 3D Reconstruction in Dynamic Environments for RGB-D Cameras Exploiting Residuals. 2019.
- [56] Federico Perazzi, Jordi Pont-Tuset, Brian McWilliams, Luc Van Gool, Markus Gross, and Alexander Sorkine-Hornung. A benchmark dataset and evaluation methodology for video object segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 724–732, 2016.
- [57] Feng Li, Hao Zhang, Huaizhe xu, Shilong Liu, Lei Zhang, Lionel M. Ni, and Heung-Yeung Shum. Mask dino: Towards a unified transformer-based framework for object detection and segmentation, 2022.
- [58] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *arXiv:2406.09414*, 2024.

ADD-SLAM: Adaptive Dynamic Dense SLAM with Gaussian Splatting

Supplementary Material

Abstract

In the document, we provide additional details about the following:

1. More information about the datasets.
2. Further implementation details.
3. Additional results and analysis.

A Datasets

TUM RGB-D dataset[54]. The TUM RGB-D Dataset offers a comprehensive collection of indoor RGB-D sequences recorded using a Microsoft Kinect sensor at 30 Hz with a resolution of 640×480. Ground-truth trajectories were captured using a high-precision motion capture system operating at 100 Hz.

Bonn dataset[55]. The Bonn RGB-D Dynamic Dataset is a dataset for RGB-D SLAM, containing highly dynamic sequences capturing human activities such as object manipulation and interaction with balloons. Each sequence includes ground-truth camera poses obtained via an OptiTrack Prime 13 motion capture system.

DAVIS Dataset[56]. The Densely Annotated Video Segmentation (DAVIS) dataset comprises high-resolution video sequences capturing a wide array of dynamic scenes and moving objects. It includes scenarios ranging from human activities and animal movements to complex object interactions and natural phenomena, providing a rich variety of motion patterns and object appearances. DAVIS doesn't contain depth information. Following WildGS-SLAM [9], we leverage *Depth Anything V2* [58] to estimate depth, ensuring compatibility with our pipeline.

B Further implementation details

Tracking settings. We employ a weighted combination of color and depth losses for tracking, with weights set to $w_{\text{color}} = 0.6$ and $w_{\text{depth}} = 0.4$, respectively. Each frame is optimized for 100 iterations. The learning rates for camera pose optimization are set to 0.002 for rotation and 0.01 for translation. To mitigate the influence of outliers, alpha and depth filtering are applied to generate masks during the computation of the tracking loss. The tracking loss is defined as:

$$\mathcal{L}_{\text{track}} = w_{\text{color}} \cdot \mathcal{L}_{\text{color}} + w_{\text{depth}} \cdot \mathcal{L}_{\text{depth}}. \quad (20)$$

Mapping settings. For mapping, the position learning rate decays from an initial value of 0.001 to a final value of 1.6×10^{-6} . The learning rates for color, opacity, scaling, and rotation are set to 0.0025, 0.05, 0.005, and 0.001, respectively. Similar to tracking, each frame undergoes 100 iterations. The opacity threshold is set to 0.8 during densification and 0.3 during pruning. The weights for the composite losses are configured as $\lambda_{\text{ssim}} = 0.2$, $\lambda_{\text{color}} = 1.0$, $\lambda_{\text{depth}} = 1.0$, and $\lambda_{\text{reg}} = 1.0$. The color loss is defined as:

$$\mathcal{L}_{\text{color}} = (1 - w_{\text{ssim}}) \cdot \mathcal{L}_{\text{rgb}} + w_{\text{ssim}} \cdot \mathcal{L}_{\text{ssim}}. \quad (21)$$

The overall tracking loss function, incorporating regularization, is:

$$\mathcal{L}_{\text{map}} = w_{\text{color}} \cdot \mathcal{L}_{\text{color}} + w_{\text{depth}} \cdot \mathcal{L}_{\text{depth}} + w_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}. \quad (22)$$

Adaptive dynamic detection and tracking settings. Adaptive dynamic object detection is performed every 5 frames for the Bonn and DAVIS datasets, and every 10 frames for the TUM RGB-D dataset. For 2D dynamic tracking, tracking is terminated if the center of a dynamic object approaches within 4% of the image boundary. Additionally, tracking is considered erroneous and is terminated if the dynamic object's mask area increases by more than 1.5× or its center moves over 20% of the field of view within a single frame.

C Additional results and analysis

Adaptive dynamic segmentation and dynamic-static separation mapping. We provide additional visualizations of the adaptive dynamic segmentation and dynamic-static separation mapping process during ADD-SLAM execution, as shown in Fig. 6 and Fig. 7. We initialize the first frame as a static Gaussian map. Then, adaptive dynamic segmentation is performed based on inconsistency detection. When a dynamic object is detected, it is separated from the static map, and a sequential dynamic Gaussian is constructed for dynamic mapping. The holes in the static map are gradually filled as the object moves.

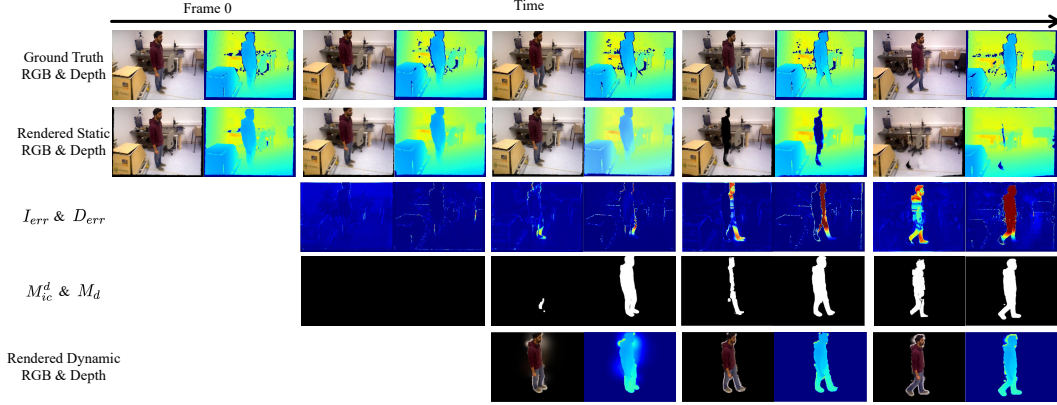


Figure 6: Visualizations of the adaptive dynamic segmentation and dynamic-static separation mapping process during ADD-SLAM execution on Scene ps_track of the Bonn dataset.

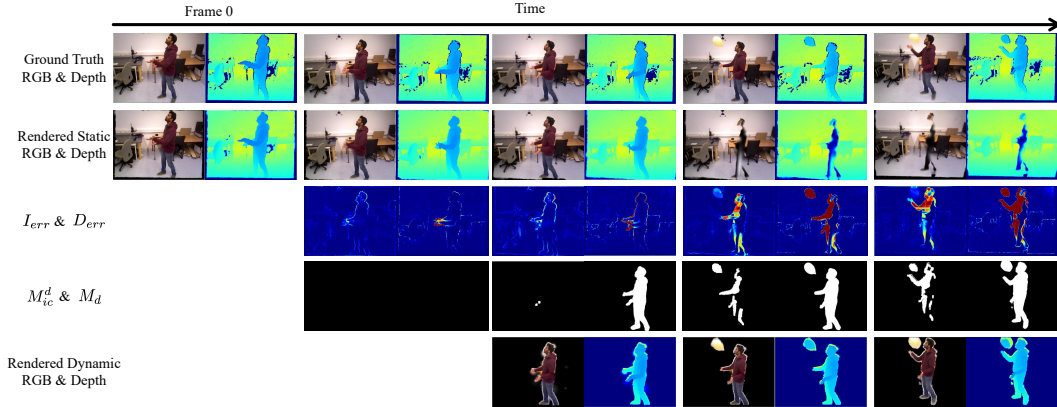


Figure 7: Visualizations of the adaptive dynamic segmentation and dynamic-static separation mapping process during ADD-SLAM execution on Scene balloon of the Bonn dataset.

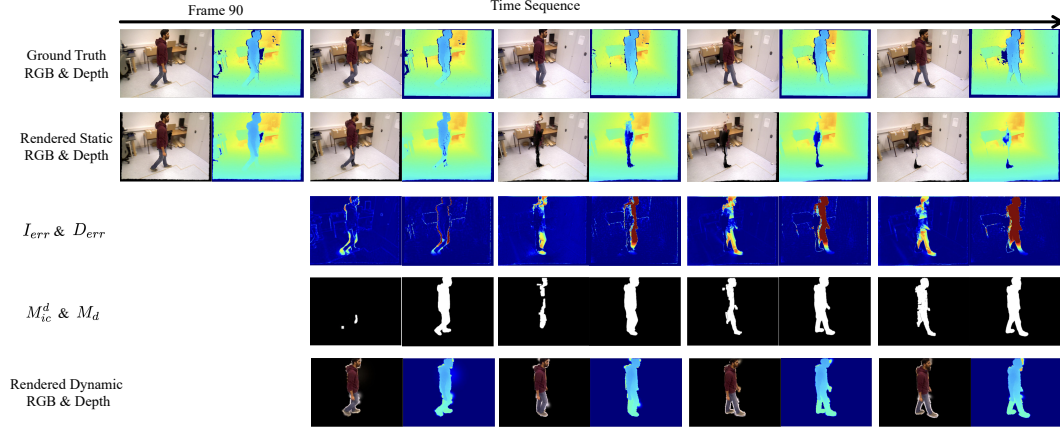


Figure 8: Visualizations of the adaptive dynamic segmentation and dynamic-static composition mapping process during ADD-SLAM execution on Scene ps_track (start at the 90th frame of the original sequence) of the Bonn dataset.

It should be noted that our method does not enforce the first frame scene to be completely static. We initialize the first frame as a static map, and the single-frame map initialization can be completed regardless of whether there are moving objects. When object movements in subsequent frames disrupt scene consistency, our method can identify moving regions and separate dynamic objects from the original map. We supplement experiments with person tracking sequences starting from the 90th frame. As shown in Fig. 8, our method remains effective when a person is in motion from the first frame. Actually, in most scenarios of our experiments, the objects have already started moving at the beginning of the sequence.

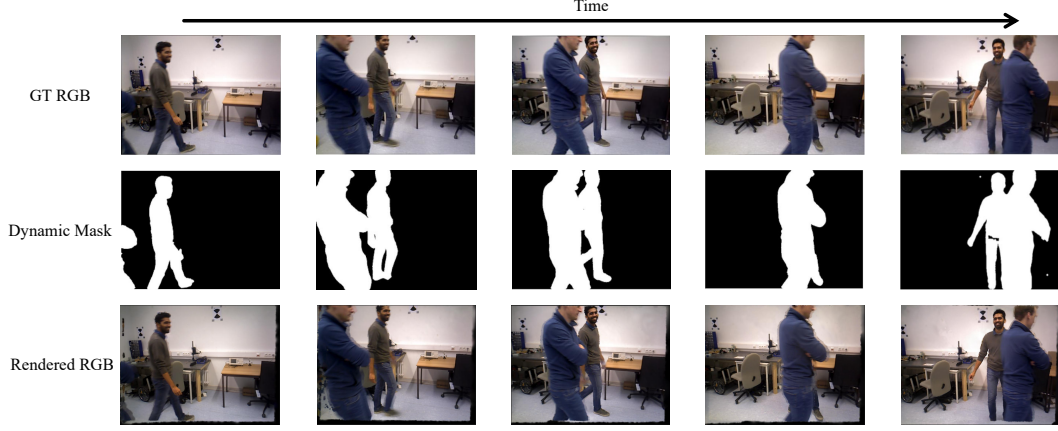


Figure 9: Visualization of dynamic mask and rendering results in Scenario crowd of the Bonn Dataset, featuring overlapping moving pedestrians.

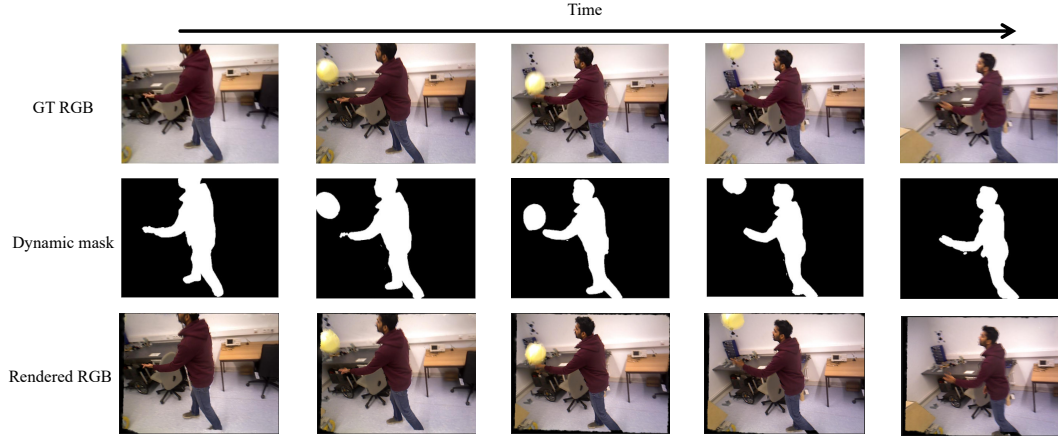


Figure 10: Visualization of dynamic mask and rendering results in the Scenario balloon of the Bonn Dataset, featuring a high-speed balloon exhibiting motion blur.

Multiple dynamic objects overlap and large movement of dynamic objects. For multiple dynamic objects that overlap, we supplement experiments on the Scenario crowd of the Bonn Dataset, containing multiple moving people, as shown in Fig. 9. For large movements of dynamic objects, we provide visualizations from the Scenario balloon, where balloon motion blur indicates high-speed movement, as shown in Fig. 10. In addition, Fig. 16 to Fig. 23 show the extensive experiments of our method on the DAVIS dataset, validating the effectiveness under challenging conditions.

Sensitivity analysis of geometric and color inconsistency thresholds. For geometric and color inconsistency thresholds, we statistically analyze the rendered geometric and color error distributions, as shown in Fig. 11 and Fig. 12. The red dashed lines indicate error medians, while the yellow shaded areas represent the range from **10x to 30x the error median**. The observed long-tailed error distribution reveals two distinct characteristics: minimal errors in static regions (distribution head) and significantly larger errors in dynamic regions (distribution tail). Selecting 20x the median as the operational threshold effectively distinguishes genuine motion areas. Notably, the error threshold maintains a wide acceptable range (10x-30x, even wider), ensuring robust applicability in practice. Fig. 13 presents a sensitivity analysis of the threshold selection, demonstrating that while the inconsistent mask area slightly decreases from 10x to 30x median thresholds. The dynamic mask segmentation results remain unaffected, with true motion regions consistently identifiable. This analysis demonstrates the effectiveness and robustness of our threshold Settings.

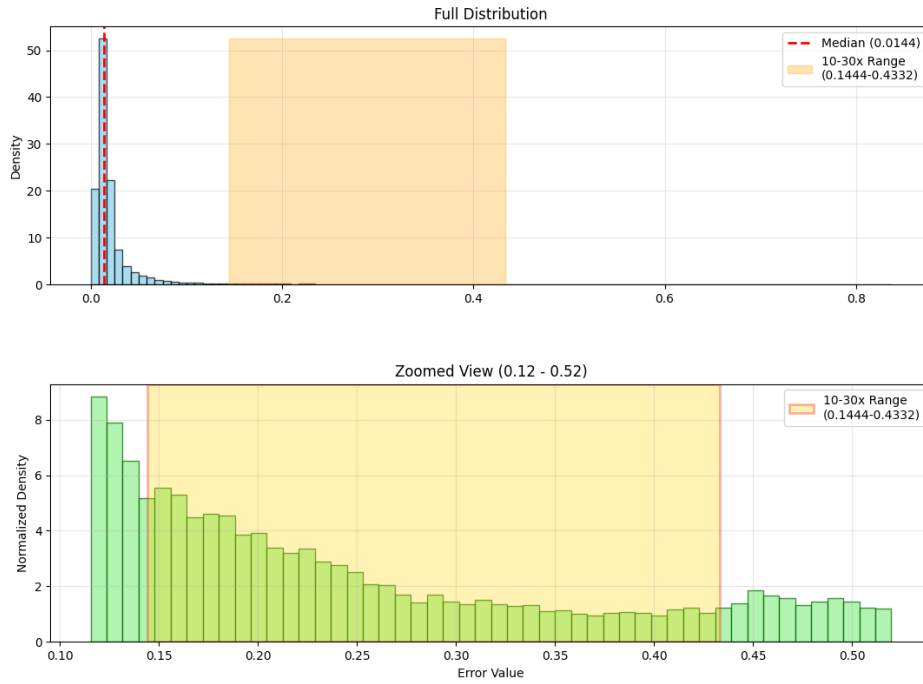


Figure 11: **RGB error distribution analysis**. The yellow part is between 10 and 30 times the median.

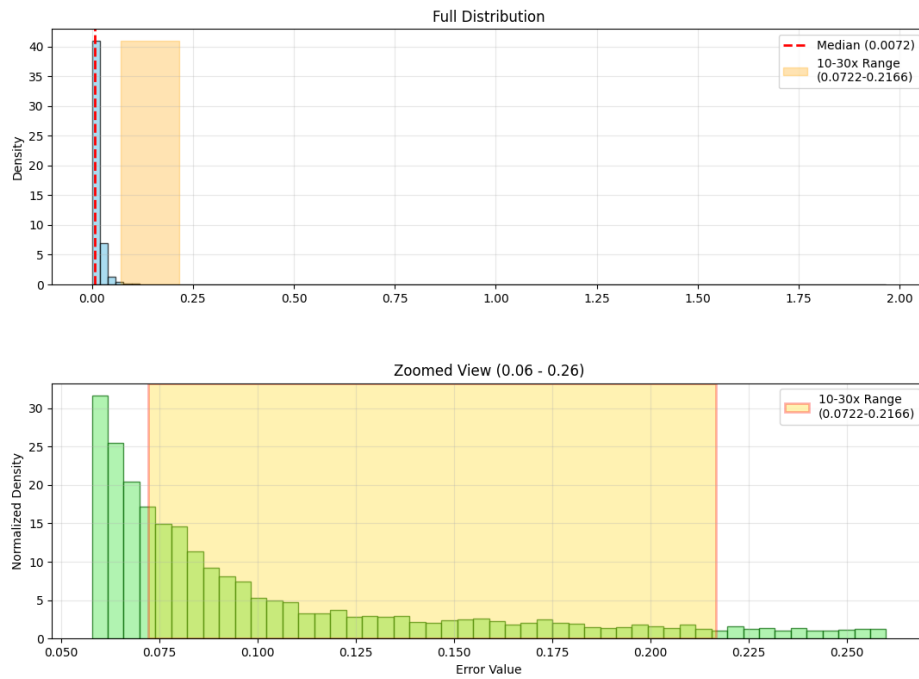


Figure 12: Depth error distribution analysis. The yellow part is between 10 and 30 times the error median.

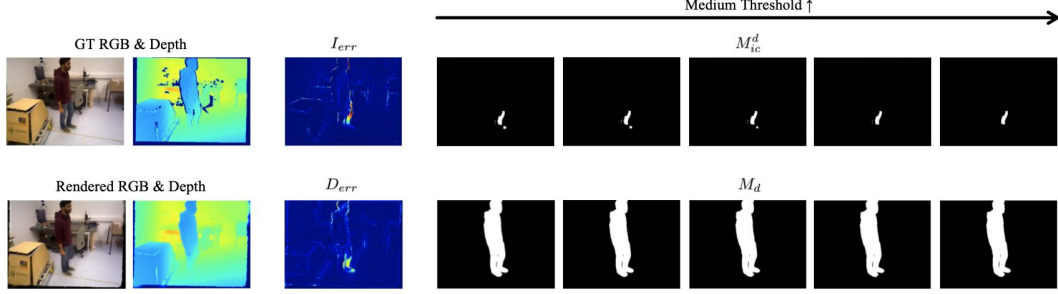


Figure 13: Inconsistent regions M_{ic}^d and dynamic masks M_d under different multiples of the error median. From left to right: 10×, 15×, 20×, 25×, and 30× the error median.

Comparison with the dynamic mask of DG-SLAM [7]. DG-SLAM [7] uses semantic segmentation prior and multi-view depth warp mask to compensate for missing objects. Even if the prior objects are static, they will be divided. Although the multi-view depth warp mask compensates for missing objects, the occlusion caused by the view change is also contained in the mask and cannot be distinguished, as shown in Fig. 14.



Figure 14: Comparison of dynamic segmentation masks between our method and DG-SLAM. The red-circled area represents the noise artifacts in the warp mask.

Supplementary results on the Davis dataset. We supplement experiments on multiple scenarios from the DAVIS dataset. Fig. 16 to Fig. 23 demonstrate our method’s performance across diverse complex scenarios, validating its robustness. The uncertainty learning in WildGS-SLAM [9] fails to effectively distinguish scene dynamics in low-motion scenarios, such as the camel shown in Fig. 16. Moreover, its uncertainty estimation is susceptible to background interference, preventing accurate localization of dynamic object regions. In contrast, our scene-consistency-based adaptive dynamic segmentation method achieves precise dynamic object masks. Furthermore, while WildGS-SLAM [9] only models static backgrounds while ignoring dynamic foregrounds, our method simultaneously models scene dynamics and preserves richer scene information.

Limitation. While our method enables adaptive discovery of dynamic regions within scenes, the integrity of dynamic object segmentation is contingent upon mobileSAM [13]. In complex environments, partial segmentation or over-segmentation may degrade system performance, as exemplified in Fig. 15(a) and (b). Furthermore, scene inconsistencies arising from imprecise depth estimation can induce erroneous dynamic object detection, such as the wrong detection of the sky in the upper-left corner of Fig. 15 (c).



Figure 15: Failure cases. The green points in the RGB image serve as prompts from inconsistency detection. From left to right, three failure cases on the DAVIS dataset are demonstrated: (a) partial segmentation, (b) over-segmentation, and (c) wrong detection caused by inaccurate depth estimation.



Figure 16: Visualization of dynamic mask and rendering results in the Scenario camel of the DAVIS Dataset.

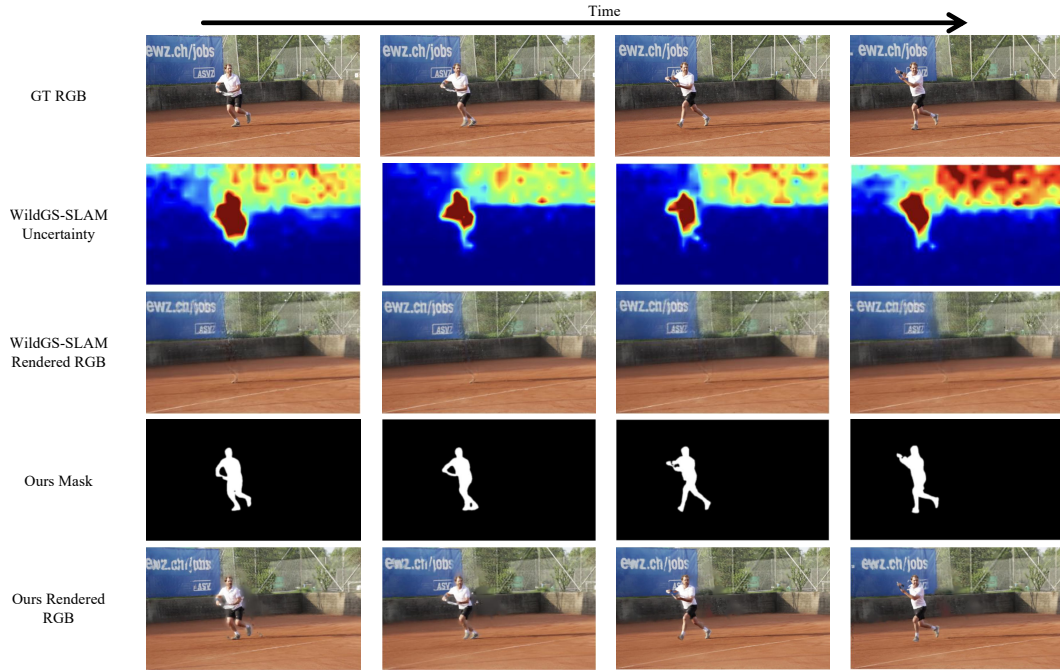


Figure 17: Visualization of dynamic mask and rendering results in the Scenario tennis of the DAVIS Dataset.

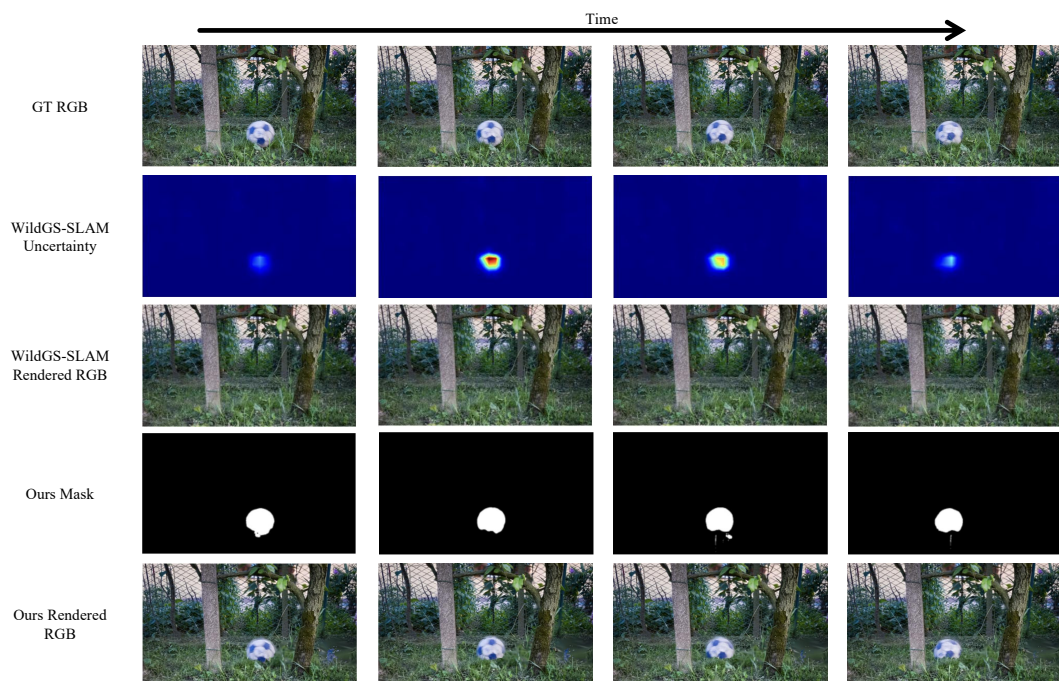


Figure 18: Visualization of dynamic mask and rendering results in the Scenario rollerblade of the DAVIS Dataset.

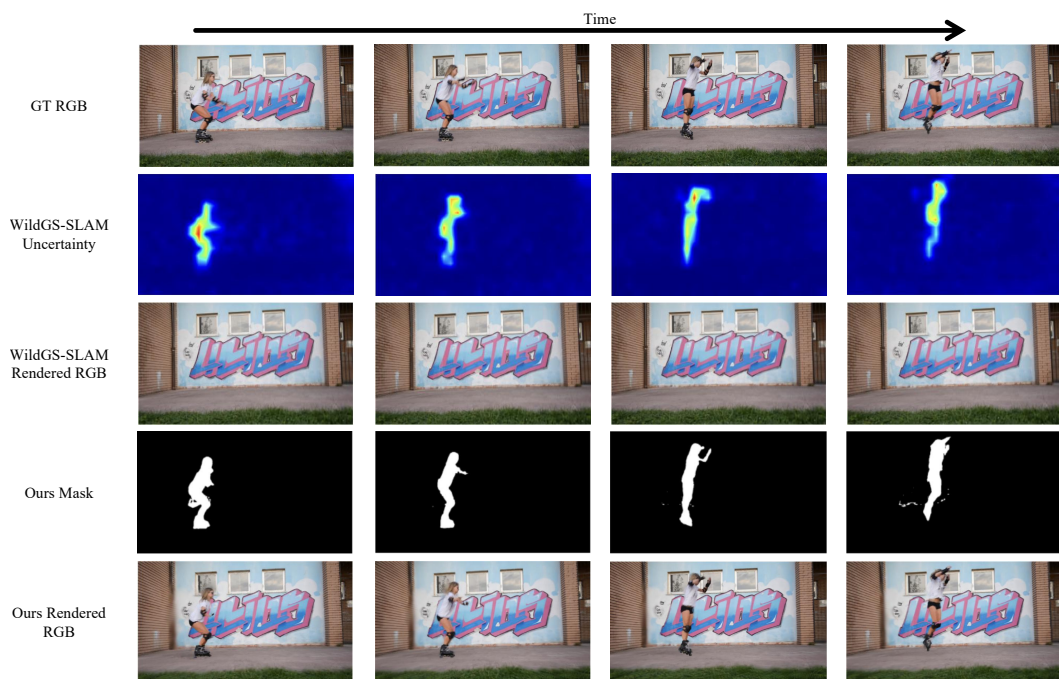


Figure 19: Visualization of dynamic mask and rendering results in the Scenario parkour of the DAVIS Dataset.

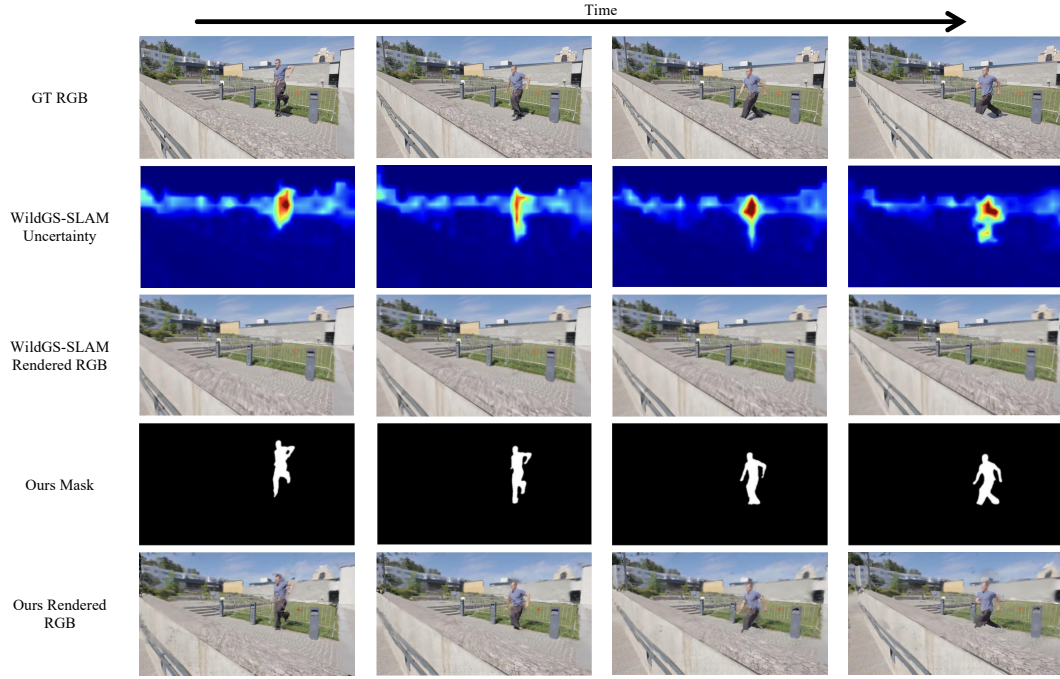


Figure 20: Visualization of dynamic mask and rendering results in the Scenario motocross-bumps of the DAVIS Dataset.

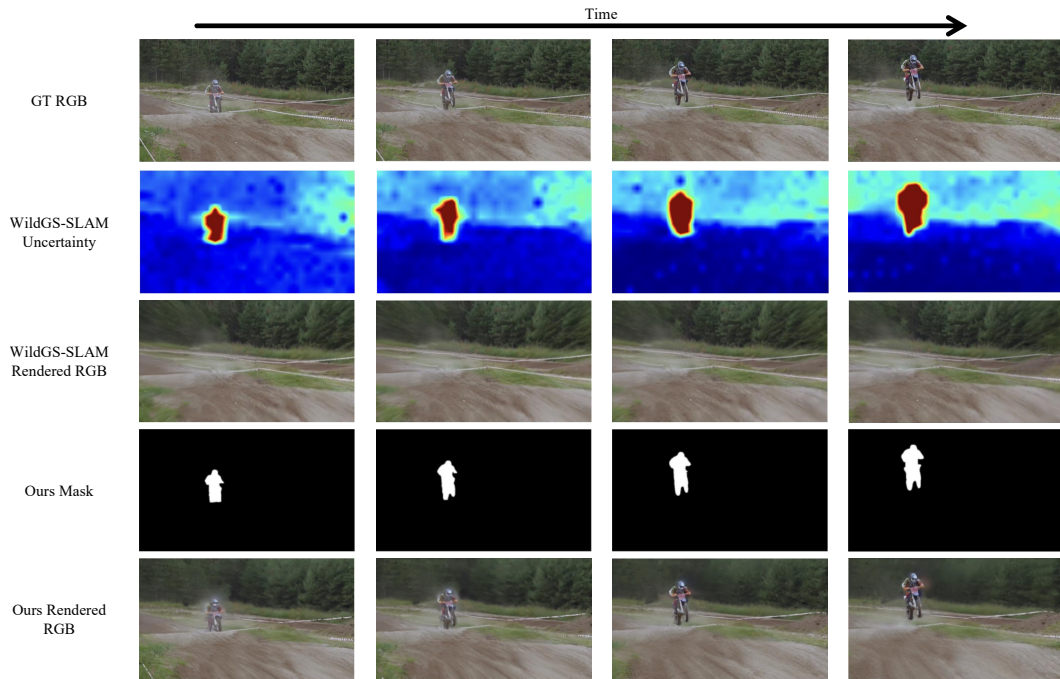


Figure 21: Visualization of dynamic mask and rendering results in the Scenario soccerball of the DAVIS Dataset.

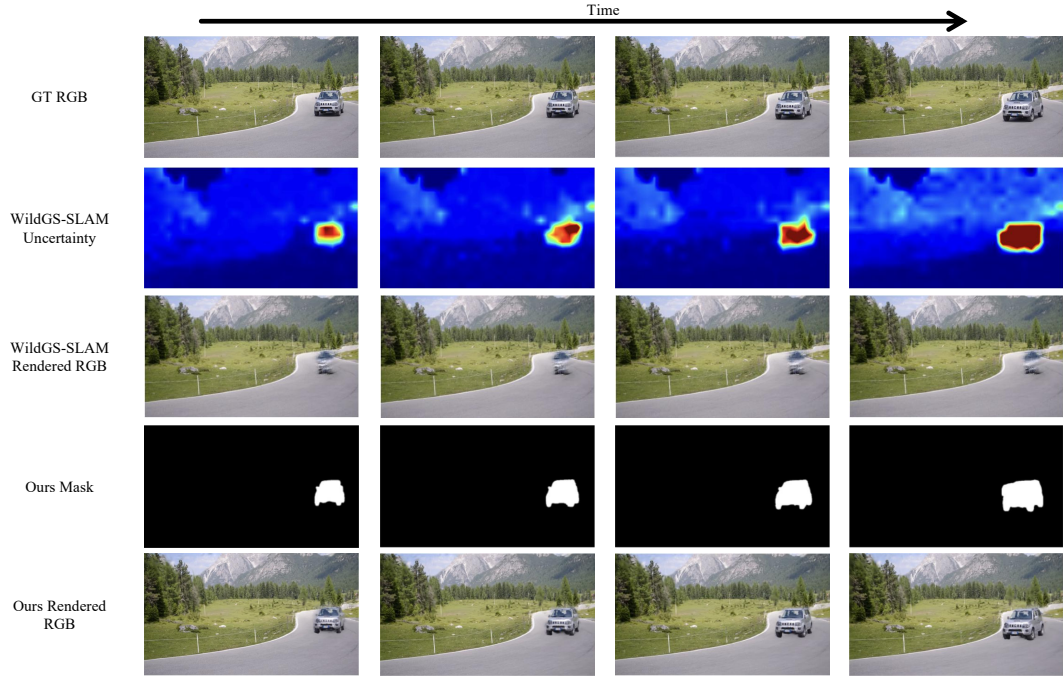


Figure 22: Visualization of dynamic mask and rendering results in the Scenario car-turn of the DAVIS Dataset.

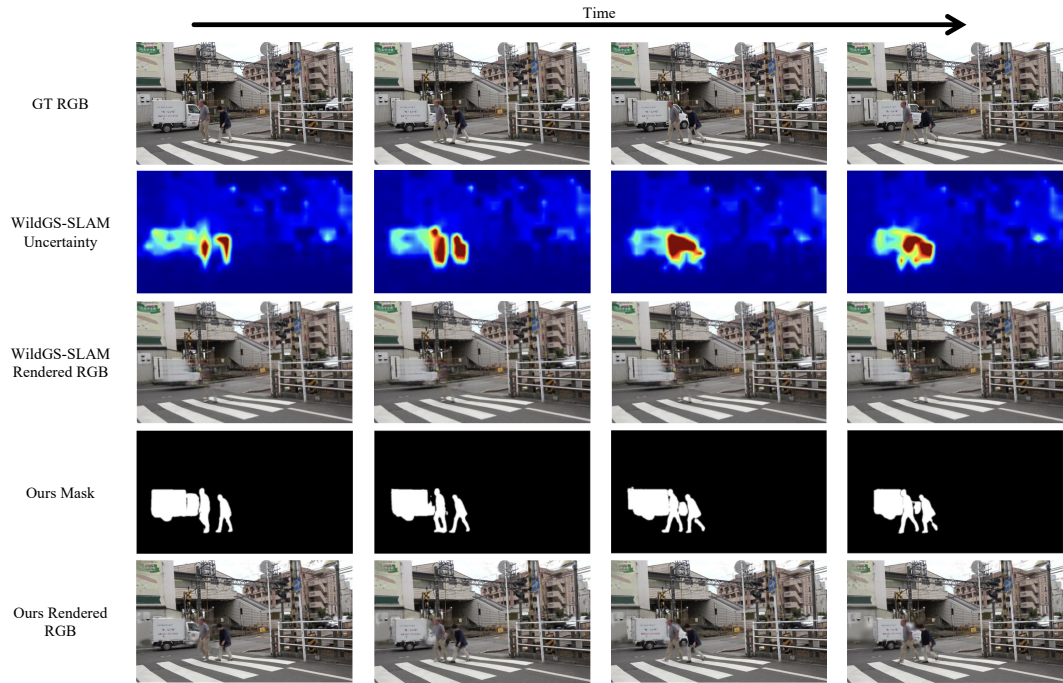


Figure 23: Visualization of dynamic mask and rendering results in the Scenario crossing of the DAVIS Dataset.